

GROUP 61 | ASSIGNMENT 02

CollabBoard

Working REST APIs with In-Memory Data

Implementation, integration, verification, and submission evidence

Technical status: Frontend and REST implementation passed automated and live integration QA. Final Git tagging remains gated on human approval and genuine commits from all five members.

NSBM Green University

Group 61

31 August 2026

1. Introduction

CollabBoard is a responsive Kanban-style collaboration workspace for small project teams. Assignment 02 connects the completed React frontend to a modular Express REST API while preserving the frontend workflows, data field names, and derived selector logic established during the frontend campaign.

The implementation uses deterministic server-side in-memory data, JWT authentication, bcrypt password hashes, explicit CORS, allowlisted validation, stable service facades, and an executable Postman collection. MongoDB, realtime features, deployment, Docker, offline synchronization, and richer collaboration features are intentionally outside this assignment boundary.

Assignment outcome: The React application now performs authentication, project CRUD, task CRUD and status movement, Team reads, and Settings updates through protected REST endpoints without direct page-level fetch calls or automatic mock fallback.

2. Team Members and Roles

Student ID	Member	Assignment role
35762	M N R Mudannayaka	Frontend, backend integration, and project coordination
35783	M D J Prabashana	Frontend components and task API verification
33223	K I S S Kariyawasm	UI/UX, responsive design, and integration verification
35934	D M S P Bandara	Dashboard, supporting UI, API verification, and QA
30824	C R Gammbheera	Common components, Postman, documentation, and Git

Contribution evidence must reflect work each person actually performed. At the time this report was generated, Git history did not yet contain verified individual commits from all five members, so the final Assignment 02 tag remains deliberately pending.

3. System Architecture



Figure 1. Assignment 02 frontend-to-backend request flow.

The stable frontend service layer is the architectural seam. Pages depend on authService, projectService, taskService, and userService. Each facade selects either the explicit mock adapter or the REST adapter. In API mode, apiClient owns JSON serialization, the Bearer header, 204 handling, normalized ApiError objects, and 401 session clearing.

- WorkspaceProvider loads users, projects, and tasks concurrently through service contracts.
- Express routes delegate to validation-aware controllers and an in-memory repository.
- Shared modules contain only public enums and public seed entities.
- Password hashes, JWT configuration, and authentication records remain backend-only.

4. Frontend-Backend Integration

The migration keeps existing root service imports stable, so page components did not need a second redesign. The REST adapters return the same entity shapes as the asynchronous mock adapters. After successful mutations, WorkspaceProvider refetches canonical collections, ensuring the UI reflects server state rather than optimistic local-only changes.

Frontend operation	HTTP request	UI result
authService.login	POST /api/auth/login	JWT session and protected dashboard
projectService.create	POST /api/projects	New project after canonical refresh
taskService.updateStatus	PATCH /api/tasks/:id/status	Task appears in the new Kanban column
userService.update	PATCH /api/users/:id	Shared current-user profile updates

5. REST API Design

The API is a small monolithic Express application organized by routes, controllers, services, repositories, middleware, data, and utilities. Success responses use bare entities or arrays. Deletions return 204 No Content. Errors use a single envelope with code, message, and details fields.

Error contract: `{"error":{"code":"TASK_NOT_FOUND","message":"Task not found","details":[]}}`

Correct HTTP semantics are used: 400 invalid input, 401 missing or invalid JWT, 403 forbidden profile update, 404 missing entity or route, 409 identity or relationship conflict, and 500 unexpected server errors.

6. Authentication

Seeded development users authenticate with the password password. The server stores only bcrypt hashes in memory. Successful login and registration return a JWT and a public user. Tokens contain sub, iat, and exp and expire after eight hours by default.

1. The Login page calls `authService.login` with email, password, and Remember Me preference.
2. The REST auth adapter posts credentials to `/api/auth/login` and stores the returned token in `localStorage` or `sessionStorage`.
3. `apiClient` adds `Authorization: Bearer <token>` to protected requests.
4. The Express middleware validates the token and attaches the public user to the request.
5. A 401 clears the browser session and returns the application to Login.

Task `creatorId` is always derived from the JWT. Client-provided creator IDs are ignored. Profile updates are self-only; project and task CRUD is shared among authenticated workspace members without adding advanced RBAC to this milestone.

7. API Endpoint Summary

Group	Public	Protected operations
Health	GET <code>/api/health</code>	None
Authentication	POST <code>/api/auth/register</code> ; POST <code>/api/auth/login</code>	GET <code>/api/auth/me</code>
Users	None	List, read, self-profile patch
Projects	None	List, read, create, patch, delete
Tasks	None	List/filter, read, create, patch, status patch, delete

Task queries accept projectId, assigneeId, status, priority, and label. Progress, workload, overdue totals, Dashboard counts, and project contribution remain frontend selector responsibilities.

8. GitHub Repository and Assignment 02 Tag

Repository: <https://github.com/RamsaraNohan/CollabBoard>

Frontend base commit: 4ac9e849c85dff5cca5a1c349771cf16b549b1a0

Implementation branch: codex/assignment02-rest-api

Planned final tag: assignment-02-working-rest-apis

Tag status: Not created. The approved process requires all five genuine member contributions and human approval of the final implementation commit before the annotated tag is created and pushed.

9. How to Run

1. Run npm install in the repository root.
2. Run npm --prefix server install for the separate backend package.
3. Copy server/.env.example to server/.env and set a local JWT_SECRET of at least 16 characters.
4. Run npm run dev:all.
5. Open <http://localhost:5173> and sign in with member1@example.com / password.

Frontend-only mock development remains explicit through npm run dev:mock. Complete commands, environment variables, testing scripts, API boundaries, and credentials are documented in README.md.

10. Postman Collection

The Postman v2.1 collection is stored at postman/CollabBoard_Assignment02.postman_collection.json. It defines baseUrl, token, userId, projectId, and taskId variables. Login captures the JWT; create operations capture temporary IDs; cleanup requests delete the temporary task and project.

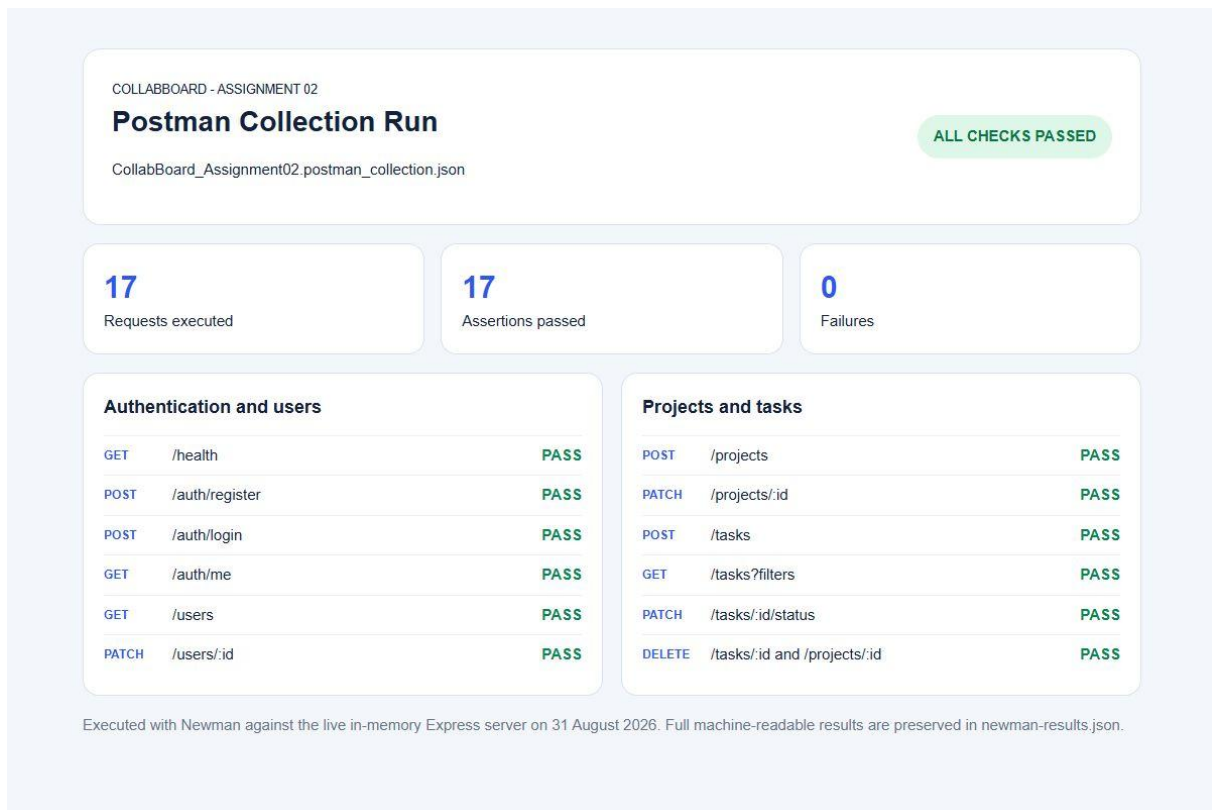


Figure 2. Executable collection result: 17 requests, 17 assertions, 0 failures.

11. Frontend Screenshots

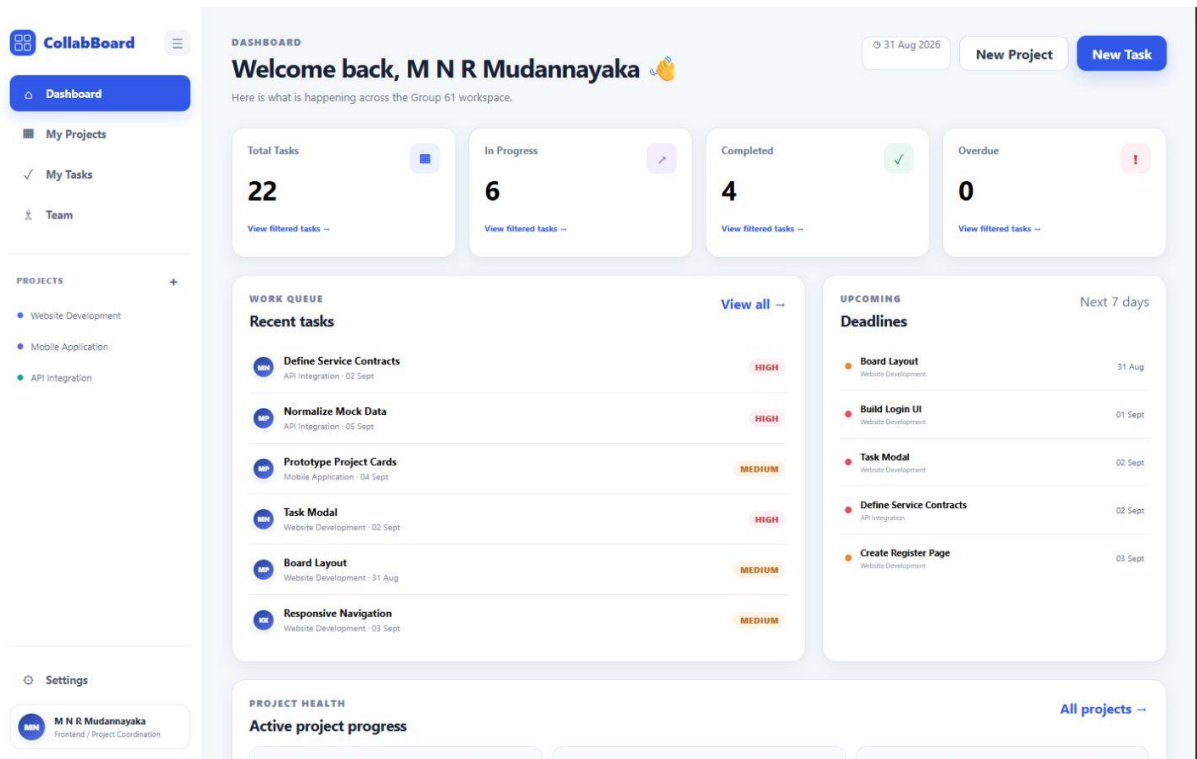


Figure 3. API-backed Dashboard with derived task counts and deadlines.

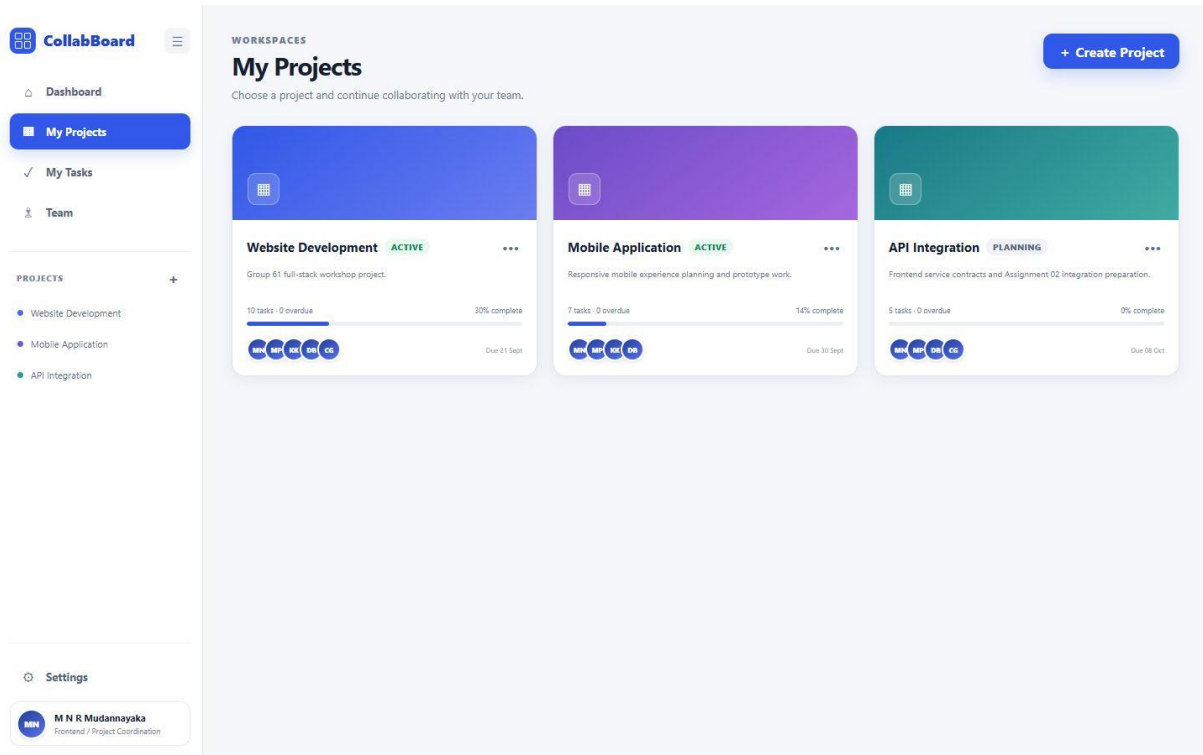


Figure 4. Canonical project list loaded from the REST API.

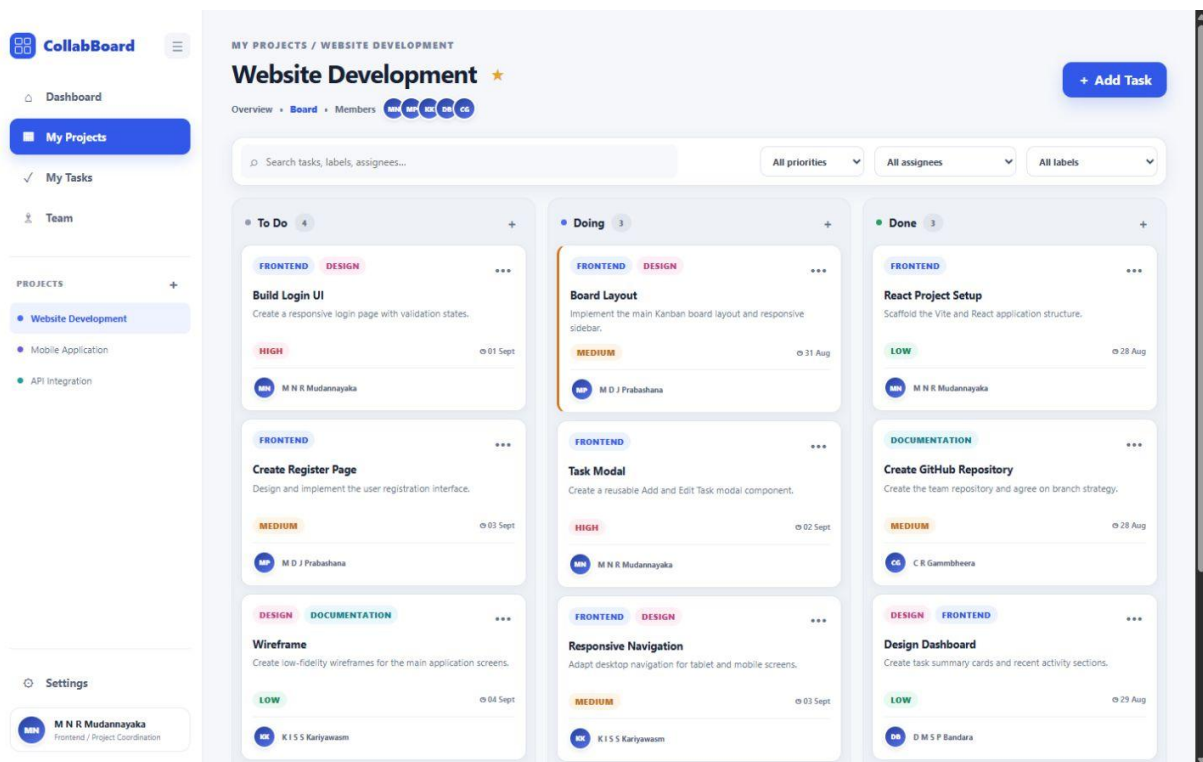


Figure 5. Project-specific Website Development Kanban board.

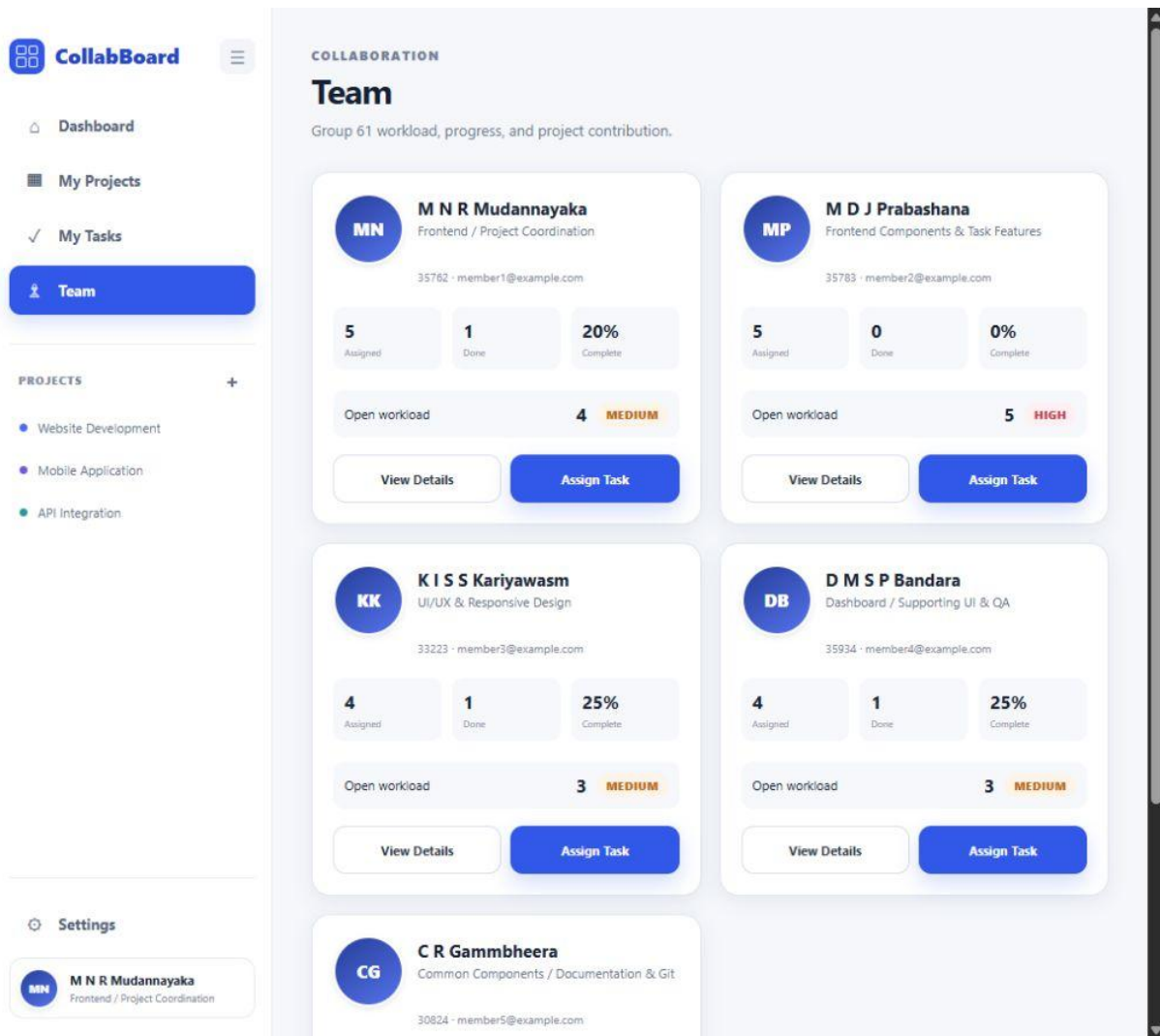


Figure 6. Five-member Team workload and completion metrics at 1024 px.

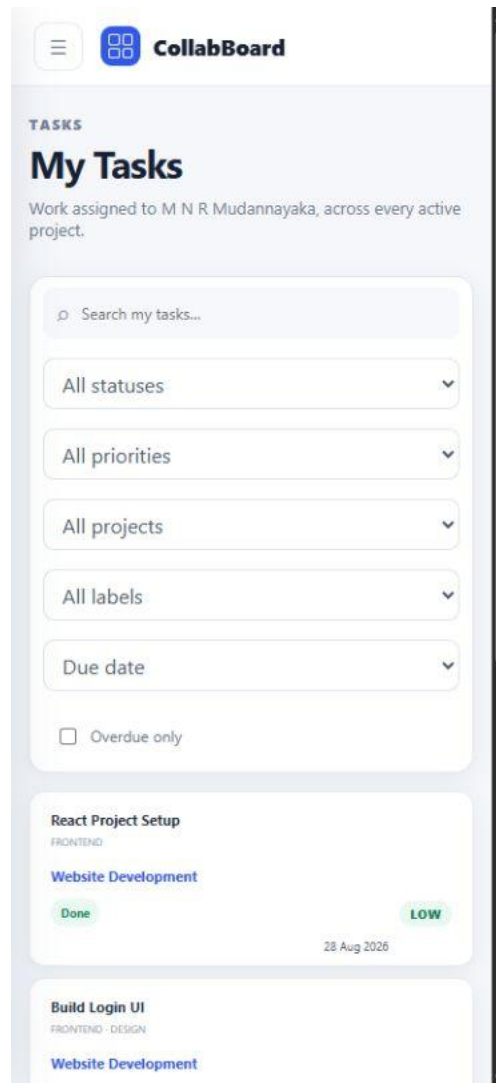


Figure 7. Mobile My Tasks card layout at 390 px.

12. Backend/API Screenshots

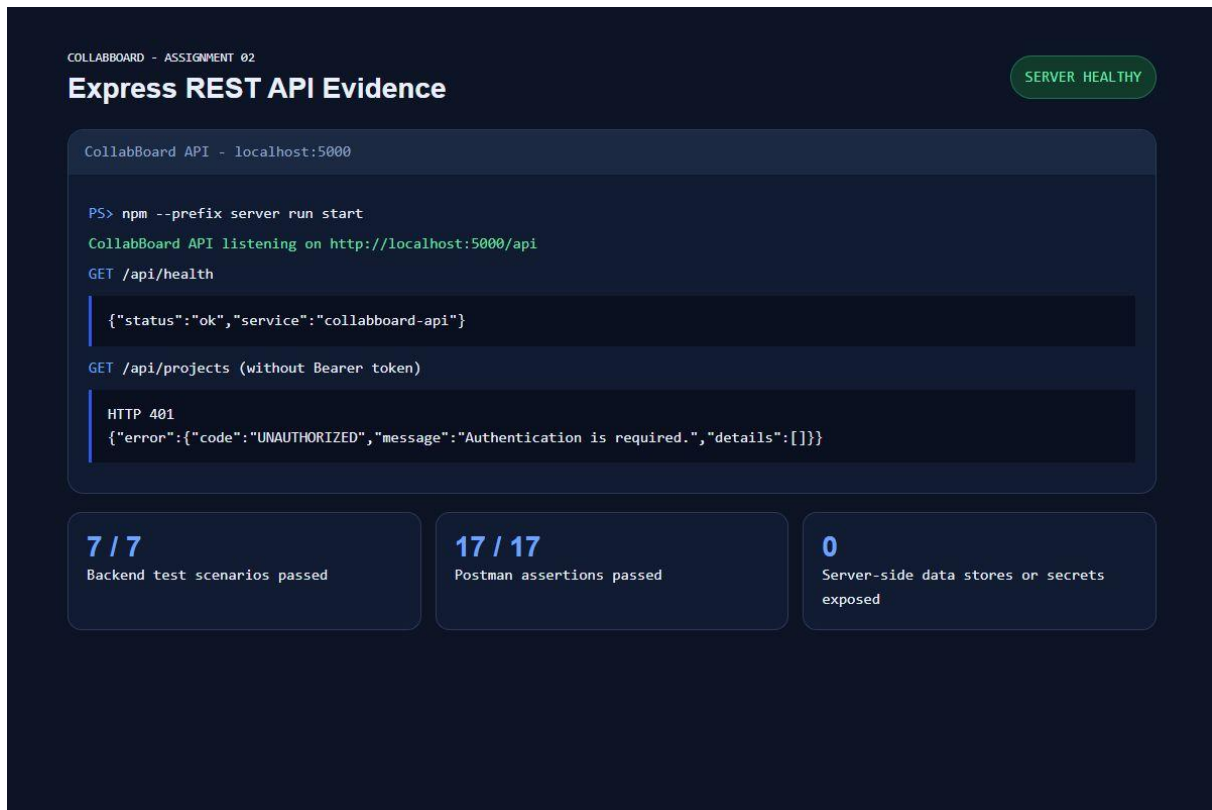


Figure 8. Express startup, health response, protected-route 401, and test summary.

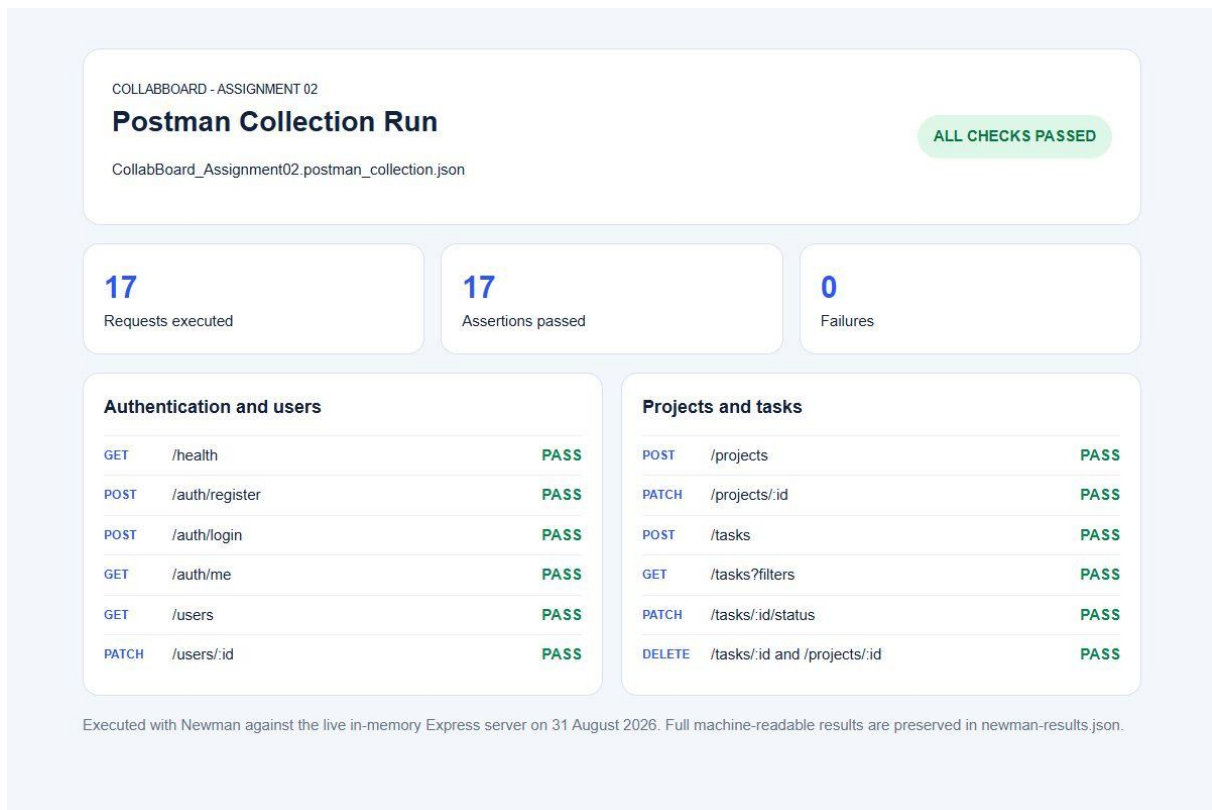


Figure 9. Postman/Newman request and assertion summary.

13. Testing and Verification

Gate	Result	Evidence
Frontend tests	PASS	7 files, 12 tests
Backend tests	PASS	1 file, 7 scenarios
Postman/Newman	PASS	17 requests, 17 assertions, 0 failures
Production build	PASS	Vite transformed 102 modules
Responsive browser QA	PASS	390, 768, 1024, and 1440 px
Console audit	PASS	Handled 409 produced zero new console errors

Live browser QA used the real API mode. It verified login and JWT restoration, invalid-token clearing, project creation, task creation/edit/status movement, project isolation, Team and Member Details, Settings synchronization, relationship conflict feedback, direct route refreshes, and mobile sidebar geometry.

Backend tests additionally verified destructive task/project cleanup, project delete cascade, invalid assignees, self-only user updates, protected routes, and normalized 400/401/403/404/409 errors.

14. Known Limitations

- In-memory mutations reset whenever the Express server restarts.
- The JWT design intentionally omits refresh tokens and account recovery for this coursework milestone.
- MongoDB, realtime updates, offline queues, comments, notifications, attachments, and deployment are deferred.
- The final Git tag is pending human approval and verified genuine contributions from all five members.
- Registered users are not automatically added to existing projects.

15. Conclusion

CollabBoard now satisfies the technical Assignment 02 objective: a completed React frontend communicates through stable asynchronous service contracts with a JWT-protected Express REST API using deterministic in-memory backend data. Core project, task, team, authentication, and settings workflows operate through HTTP while derived presentation metrics remain correctly owned by frontend selectors.

The implementation has passed automated frontend and backend tests, an executable Postman collection, production build, live integration checks, responsive browser QA, and visual evidence review. Submission tagging must wait for the remaining people-dependent contribution and approval gates; no artificial commits or premature tag are included.